TP 3 : Gestion d'une résidence universitaire

Rapport de TP – Programmation Orientée Objet en C++

Nom et prénoms :	.....
Filière / Classe :	.....
Enseignant :	.....
Date :	13 novembre 2026

1) Introduction

Une résidence universitaire doit gérer ses chambres, ses occupants, les demandes d'hébergement et les périodes d'occupation. L'objectif de ce TP est de réaliser une application en C++ qui permet au responsable de suivre les attributions de chambres et d'**éviter les affectations multiples**. La modélisation UML (cas d'utilisation, classes, séquence, états-transitions) est fournie dans le document joint *TP3_Residence_Modelisation_UML.pdf*.

2) Analyse du besoin

Chaque fonctionnalité attendue correspond à une méthode de la classe `Residence` :

N°	Fonctionnalité	Méthode
1	Enregistrer les demandes d'hébergement	`enregistrerDemande()`
2	Attribuer une chambre à un étudiant	`attribuerChambre()`
3	Vérifier les places disponibles	`afficherPlaces()`
4	Enregistrer les départs	`enregistrerDepart()`
5	Consulter les chambres occupées et libres	`afficherChambres()`
6	Consulter l'historique des occupations	`afficherHistorique()`
Ext.	Taux d'occupation et statistiques	`afficherStatistiques()`

Règles de gestion respectées par le programme :

- un étudiant ne peut pas avoir deux demandes en attente, ni refaire une demande s'il est déjà logé ;
- une chambre ne peut pas dépasser sa capacité (pas d'attribution dans une chambre pleine) ;
- seule une demande en attente peut être acceptée, refusée ou annulée ;
- un départ libère la place et clôture l'attribution, qui reste dans l'historique.

3) Conception : les classes

Classe	Rôle
Etudiant	Représente un étudiant (matricule, nom, prénom)
Chambre	Numéro, capacité et nombre d'occupants ; méthodes `occuper()` et `liberer()`
Batiment	Contient un ensemble de chambres ; calcule sa capacité et son nombre d'occupants
DemandeHebergement	Une demande d'un étudiant, avec son statut (énumération `Statut`)
Attribution	Lie un étudiant à une chambre avec une date de début et de fin (ligne de l'historique)
Residence	Classe principale : contient les autres classes et réalise toutes les fonctionnalités

Tous les attributs sont **privés** (encapsulation). La résidence est **composée** de bâtiments, d'étudiants, de demandes et d'attributions, et un bâtiment est composé de chambres. Chaque méthode vérifie les règles de gestion *avant* de modifier les données, ce qui évite toute affectation multiple.

4) Code source

Fichier `residence.cpp` (compilation : `g++ -std=c++11 -Wall -Wextra residence.cpp -o residence`).

```
// ============================================================
//  TP 3 - Gestion d'une residence universitaire
// ============================================================
#include <iostream>
#include <iomanip>
#include <string>
#include <vector>
using namespace std;

// Etats possibles d'une demande d'hebergement
enum Statut { EN_ATTENTE, ACCEPTEE, REFUSEE, ANNULEE, TERMINEE };

string texteStatut(Statut s) {
    string noms[] = {"EN_ATTENTE", "ACCEPTEE", "REFUSEE", "ANNULEE", "TERMINEE"};
    return noms[s];
}

// ---------------------- Classe Etudiant ----------------------
class Etudiant {
    string matricule, nom, prenom;
public:
    Etudiant(string m, string n, string p) : matricule(m), nom(n), prenom(p) {}
    string getMatricule() const { return matricule; }
    string getNomComplet() const { return nom + " " + prenom; }
};

// ---------------------- Classe Chambre ----------------------
class Chambre {
    int numero, capacite, occupants;
public:
    Chambre(int n, int c) : numero(n), capacite(c), occupants(0) {}
    int getNumero() const { return numero; }
    int getCapacite() const { return capacite; }
    int getOccupants() const { return occupants; }
    int placesLibres() const { return capacite - occupants; }
    void occuper() { occupants++; }
    void liberer() { occupants--; }
};

// ---------------------- Classe Batiment ----------------------
class Batiment {
    string nom;
    vector<Chambre> chambres;
```

```
public:
    Batiment(string n) : nom(n) {}
    string getNom() const { return nom; }
    const vector<Chambre>& getChambres() const { return chambres; }

    void ajouterChambre(int numero, int capacite) {
        chambres.push_back(Chambre(numero, capacite));
    }
    Chambre* trouverChambre(int numero) {
        for (Chambre& c : chambres)
            if (c.getNumero() == numero) return &c;
        return nullptr;
    }
    int capacite() const {
        int total = 0;
        for (const Chambre& c : chambres) total += c.getCapacite();
        return total;
    }
    int occupants() const {
        int total = 0;
        for (const Chambre& c : chambres) total += c.getOccupants();
        return total;
    }
};

// ------------------ Classe DemandeHebergement ----------------
class DemandeHebergement {
    int id;
    string matricule;
    Statut statut;
public:
    DemandeHebergement(int i, string m) : id(i), matricule(m), statut(EN_ATTENTE) {}
    int getId() const { return id; }
    string getMatricule() const { return matricule; }
    Statut getStatut() const { return statut; }
    void setStatut(Statut s) { statut = s; }
};

// -------------------- Classe Attribution ---------------------
class Attribution {
    string matricule, batiment;
    int chambre;
    string debut, fin;
    bool active;
public:
    Attribution(string m, string b, int c, string d)
        : matricule(m), batiment(b), chambre(c), debut(d), fin(""), active(true) {}
    string getMatricule() const { return matricule; }
    string getBatiment() const { return batiment; }
    int getChambre() const { return chambre; }
    bool estActive() const { return active; }
    void cloturer(string date) { fin = date; active = false; }
    void afficher() const {
        cout << "  " << matricule << " | " << batiment << " ch." << chambre
             << " | du " << debut << (active ? " -> en cours" : " au " + fin) << endl;
    }
};

// --------------------- Classe Residence ----------------------
class Residence {
    string nom;
    vector<Batiment> batiments;
    vector<Etudiant> etudiants;
    vector<DemandeHebergement> demandes;
    vector<Attribution> attributions;
    int prochainId;

    // ----- methodes de recherche -----
    bool etudiantExiste(string matricule) const {
        for (const Etudiant& e : etudiants)
            if (e.getMatricule() == matricule) return true;
        return false;
    }
    DemandeHebergement* trouverDemande(int id) {
        for (DemandeHebergement& d : demandes)
            if (d.getId() == id) return &d;
        return nullptr;
    }
    Batiment* trouverBatiment(string nomBat) {
```

```
        for (Batiment& b : batiments)
            if (b.getNom() == nomBat) return &b;
        return nullptr;
    }
    Attribution* attributionActive(string matricule) {
        for (Attribution& a : attributions)
            if (a.estActive() && a.getMatricule() == matricule) return &a;
        return nullptr;
    }
    // Refuser ou annuler : seule une demande EN_ATTENTE peut changer d'etat
    void traiterDemande(int id, Statut nouveau) {
        DemandeHebergement* d = trouverDemande(id);
        if (d == nullptr || d->getStatut() != EN_ATTENTE) {
            cout << "  Erreur : demande #" << id << " introuvable ou deja traitee" << endl;
            return;
        }
        d->setStatut(nouveau);
        cout << "  Demande #" << id << " : " << texteStatut(nouveau) << endl;
    }

public:
    Residence(string n) : nom(n), prochainId(1) {}
    void ajouterBatiment(const Batiment& b) { batiments.push_back(b); }
    void ajouterEtudiant(const Etudiant& e) { etudiants.push_back(e); }

    // 1) Enregistrer une demande d'hebergement
    void enregistrerDemande(string matricule) {
        if (!etudiantExiste(matricule)) {
            cout << "  Erreur : etudiant " << matricule << " inconnu" << endl;
            return;
        }
        if (attributionActive(matricule) != nullptr) {
            cout << "  Erreur : " << matricule << " a deja une chambre" << endl;
            return;
        }
        for (const DemandeHebergement& d : demandes)
            if (d.getMatricule() == matricule && d.getStatut() == EN_ATTENTE) {
                cout << "  Erreur : " << matricule << " a deja une demande en attente" << endl;
                return;
            }
        demandes.push_back(DemandeHebergement(prochainId, matricule));
        cout << "  Demande #" << prochainId << " enregistree pour " << matricule << endl;
        prochainId++;
    }

    // 2) Attribuer une chambre (toutes les verifications avant de modifier)
    void attribuerChambre(int idDemande, string nomBat, int numChambre, string date) {
        DemandeHebergement* d = trouverDemande(idDemande);
        if (d == nullptr || d->getStatut() != EN_ATTENTE) {
            cout << "  Erreur : demande #" << idDemande << " introuvable ou deja traitee" << endl;
            return;
        }
        if (attributionActive(d->getMatricule()) != nullptr) {
            cout << "  Erreur : l'etudiant a deja une chambre" << endl;
            return;
        }
        Batiment* b = trouverBatiment(nomBat);
        Chambre* c = (b == nullptr) ? nullptr : b->trouverChambre(numChambre);
        if (c == nullptr) {
            cout << "  Erreur : chambre introuvable" << endl;
            return;
        }
        if (c->placesLibres() == 0) {
            cout << "  Erreur : la chambre " << nomBat << " " << numChambre << " est pleine" << endl;
            return;
        }
        c->occuper();
        attributions.push_back(Attribution(d->getMatricule(), nomBat, numChambre, date));
        d->setStatut(ACCEPTEE);
        cout << "  Demande #" << idDemande << " acceptee : " << nomBat
             << ", chambre " << numChambre << endl;
    }

    void refuserDemande(int id) { traiterDemande(id, REFUSEE); }
    void annulerDemande(int id) { traiterDemande(id, ANNULEE); }

    // 4) Enregistrer un depart
    void enregistrerDepart(string matricule, string date) {
        Attribution* a = attributionActive(matricule);
```

```
        if (a == nullptr) {
            cout << "  Erreur : " << matricule << " n'est pas loge" << endl;
            return;
        }
        trouverBatiment(a->getBatiment())->trouverChambre(a->getChambre())->liberer();
        a->cloturer(date);
        for (DemandeHebergement& d : demandes)
            if (d.getMatricule() == matricule && d.getStatut() == ACCEPTEE)
                d.setStatut(TERMINEE);
        cout << "  Depart de " << matricule << " enregistre (" << date << ")" << endl;
    }

    // 3) Verifier les places disponibles
    void afficherPlaces() const {
        int total = 0;
        for (const Batiment& b : batiments) {
            int libres = b.capacite() - b.occupants();
            cout << "  " << b.getNom() << " : " << libres << " place(s) libre(s) sur "
                 << b.capacite() << endl;
            total += libres;
        }
        cout << "  Total : " << total << " place(s) disponible(s)" << endl;
    }

    // 5) Consulter les chambres occupees et libres
    void afficherChambres() const {
        cout << " Chambres occupees :" << endl;
        for (const Batiment& b : batiments)
            for (const Chambre& c : b.getChambres())
                if (c.getOccupants() > 0)
                    cout << "  " << b.getNom() << " ch." << c.getNumero() << " : "
                         << c.getOccupants() << "/" << c.getCapacite() << endl;
        cout << " Chambres libres :" << endl;
        for (const Batiment& b : batiments)
            for (const Chambre& c : b.getChambres())
                if (c.getOccupants() == 0)
                    cout << "  " << b.getNom() << " ch." << c.getNumero()
                         << " (" << c.getCapacite() << " place(s))" << endl;
    }

    // 6) Consulter l'historique des occupations
    void afficherHistorique() const {
        for (const Attribution& a : attributions) a.afficher();
    }

    // Extension : taux d'occupation et statistiques
    void afficherStatistiques() const {
        int capacite = 0, occupants = 0;
        cout << fixed << setprecision(1);
        for (const Batiment& b : batiments) {
            capacite += b.capacite();
            occupants += b.occupants();
            cout << "  " << b.getNom() << " : " << b.occupants() << "/" << b.capacite()
                 << " -> " << 100.0 * b.occupants() / b.capacite() << " %" << endl;
        }
        double taux = (capacite == 0) ? 0 : 100.0 * occupants / capacite;
        cout << "  Residence : " << occupants << "/" << capacite
             << " -> taux d'occupation = " << taux << " %" << endl;
        cout << "  Demandes :";
        for (int s = EN_ATTENTE; s <= TERMINEE; s++) {
            int n = 0;
            for (const DemandeHebergement& d : demandes)
                if (d.getStatut() == s) n++;
            cout << " " << texteStatut((Statut)s) << "=" << n;
        }
        cout << endl;
    }
};

// ------------------------ Programme ------------------------
int main() {
    Residence res("Residence du Campus");

    Batiment blocA("Bloc A"), blocB("Bloc B");
    blocA.ajouterChambre(101, 2);
    blocA.ajouterChambre(102, 1);
    blocB.ajouterChambre(201, 2);
    blocB.ajouterChambre(202, 1);
    res.ajouterBatiment(blocA);
```

```
    res.ajouterBatiment(blocB);

    res.ajouterEtudiant(Etudiant("E001", "Adjovi", "Koffi"));
    res.ajouterEtudiant(Etudiant("E002", "Dossou", "Mariam"));
    res.ajouterEtudiant(Etudiant("E003", "Hounkpatin", "Sylvie"));
    res.ajouterEtudiant(Etudiant("E004", "Agbodjan", "Serge"));
    res.ajouterEtudiant(Etudiant("E005", "Gbaguidi", "Rodrigue"));
    res.ajouterEtudiant(Etudiant("E006", "Tossou", "Estelle"));

    cout << "=== 1. Demandes d'hebergement ===" << endl;
    res.enregistrerDemande("E001");
    res.enregistrerDemande("E002");
    res.enregistrerDemande("E003");
    res.enregistrerDemande("E004");
    res.enregistrerDemande("E005");
    res.enregistrerDemande("E006");
    res.enregistrerDemande("E001");   // doublon : refuse

    cout << "\n=== 2. Places disponibles ===" << endl;
    res.afficherPlaces();

    cout << "\n=== 3. Attribution des chambres ===" << endl;
    res.attribuerChambre(1, "Bloc A", 101, "2026-09-15");
    res.attribuerChambre(2, "Bloc A", 101, "2026-09-15");
    res.attribuerChambre(3, "Bloc A", 102, "2026-09-16");
    res.attribuerChambre(4, "Bloc B", 201, "2026-09-16");
    res.attribuerChambre(5, "Bloc A", 101, "2026-09-17");   // chambre pleine
    res.attribuerChambre(1, "Bloc B", 202, "2026-09-17");   // deja acceptee
    res.refuserDemande(6);

    cout << "\n=== 4. Chambres occupees et libres ===" << endl;
    res.afficherChambres();

    cout << "\n=== 5. Departs ===" << endl;
    res.enregistrerDepart("E002", "2026-12-20");
    res.enregistrerDepart("E006", "2026-12-20");   // jamais loge
    res.attribuerChambre(5, "Bloc A", 101, "2026-12-21");
    res.afficherPlaces();

    cout << "\n=== 6. Historique des occupations ===" << endl;
    res.afficherHistorique();

    cout << "\n=== 7. Statistiques ===" << endl;
    res.afficherStatistiques();

    return 0;
}
```

5) Résultats de l'exécution

```
=== 1. Demandes d'hebergement ===
  Demande #1 enregistree pour E001
  Demande #2 enregistree pour E002
  Demande #3 enregistree pour E003
  Demande #4 enregistree pour E004
  Demande #5 enregistree pour E005
  Demande #6 enregistree pour E006
  Erreur : E001 a deja une demande en attente

=== 2. Places disponibles ===
  Bloc A : 3 place(s) libre(s) sur 3
  Bloc B : 3 place(s) libre(s) sur 3
  Total : 6 place(s) disponible(s)

=== 3. Attribution des chambres ===
  Demande #1 acceptee : Bloc A, chambre 101
  Demande #2 acceptee : Bloc A, chambre 101
  Demande #3 acceptee : Bloc A, chambre 102
  Demande #4 acceptee : Bloc B, chambre 201
  Erreur : la chambre Bloc A 101 est pleine
  Erreur : demande #1 introuvable ou deja traitee
  Demande #6 : REFUSEE

=== 4. Chambres occupees et libres ===
 Chambres occupees :
  Bloc A ch.101 : 2/2
  Bloc A ch.102 : 1/1
  Bloc B ch.201 : 1/2
 Chambres libres :
  Bloc B ch.202 (1 place(s))

=== 5. Departs ===
  Depart de E002 enregistre (2026-12-20)
  Erreur : E006 n'est pas loge
  Demande #5 acceptee : Bloc A, chambre 101
  Bloc A : 0 place(s) libre(s) sur 3
  Bloc B : 2 place(s) libre(s) sur 3
  Total : 2 place(s) disponible(s)

=== 6. Historique des occupations ===
  E001 | Bloc A ch.101 | du 2026-09-15 -> en cours
  E002 | Bloc A ch.101 | du 2026-09-15 au 2026-12-20
  E003 | Bloc A ch.102 | du 2026-09-16 -> en cours
  E004 | Bloc B ch.201 | du 2026-09-16 -> en cours
  E005 | Bloc A ch.101 | du 2026-12-21 -> en cours

=== 7. Statistiques ===
  Bloc A : 3/3 -> 100.0 %
  Bloc B : 1/3 -> 33.3 %
  Residence : 4/6 -> taux d'occupation = 66.7 %
  Demandes : EN_ATTENTE=0 ACCEPTEE=4 REFUSEE=1 ANNULEE=0 TERMINEE=1
```

Commentaire : les demandes en doublon, l'attribution dans une chambre pleine, la ré-attribution d'une demande déjà acceptée et le départ d'un étudiant non logé sont tous refusés avec un message d'erreur. Après le départ de E002, la place libérée en chambre 101 est réattribuée à E005.

6) Extension : taux d'occupation

Le taux d'occupation se calcule par la formule `taux = occupants / capacité × 100`. Dans l'exécution ci-dessus, la résidence compte 4 occupants pour 6 places, soit **66,7 %** (Bloc A : 3/3 = 100 % ; Bloc B : 1/3 = 33,3 %). La méthode `afficherStatistiques()` affiche aussi le nombre de demandes par statut.

7) Conclusion

Ce TP nous a permis de modéliser un système de gestion avec UML puis de l'implémenter en C++ en appliquant l'encapsulation, la composition et l'association entre classes. L'application répond à toutes les fonctionnalités demandées et empêche les affectations multiples. Elle pourrait être améliorée par une sauvegarde des données dans un fichier et un menu interactif.